LAPORAN PRATIKUM

Pemrograman Berbasis Objek

disusun Oleh:

Muhammad Yasin Habiburrahman

2511532016

Praktikum Kelas B

Dosen Pengampu:

Afdhal Dinilhak S.Kom, M.Kom

Asisten Pratikum:

Siti Fadhilah Rahmi



DEPARTEMEN INFORMATIKA FAKULTAS TEKNOLOGI INFORMASI

UNIVERSITAS ANDALAS

TAHUN 2026

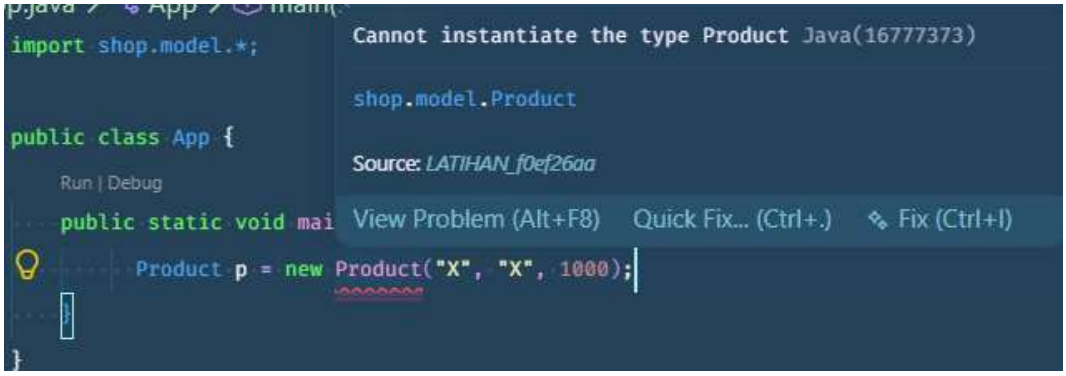
LATIHAN PRAKTIKUM **1**
Error Compiler 1
L1.e 1
L3 1
L4 2
T1a. 2
T1.e 4
T1 4
T2 4
T3 6
T4 7
SCREENSHOTS **9**

LATIHAN PRAKTIKUM

- NIM: 2511532016
- CARD_ADMIN_PERCENT = 1
- TRANSFER_ADMIN_FEE = 3000
- SMS_MAX_LENGTH = 116

Error Compiler

L1.e



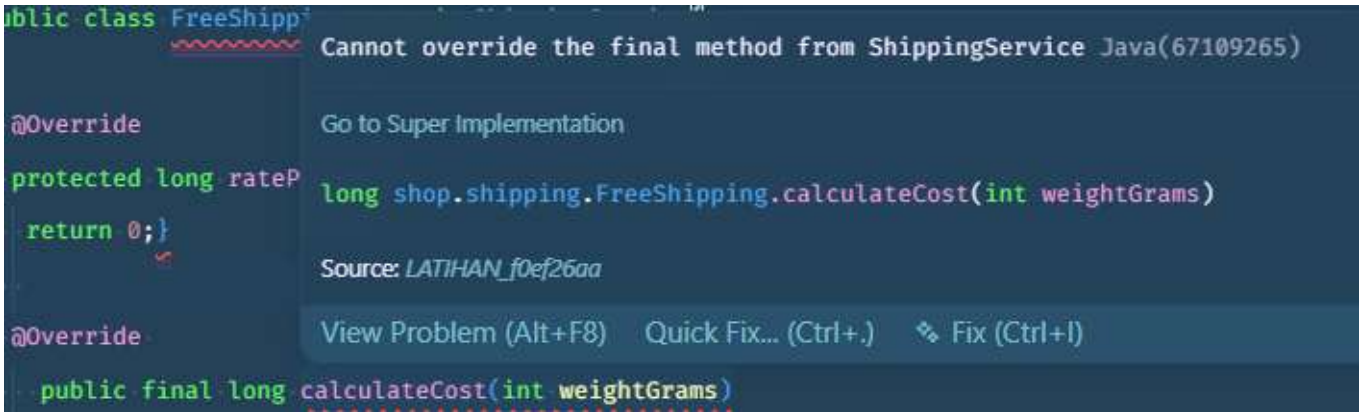
Kelas Product merupakan kelas abstract. Kita tidak bisa membuat instance secara langsung dari kelas abstract

L3



Method refund pada kelas CheckoutService hanya menerima parameter dengan instance RefundablePayment. BankTransferPayement hanya mengimplementasikan PaymentMethod.

L4



Method calculateCost adalah method final. Artinya isi/body dari method tersebut tidak boleh diubah (override) oleh sub-classnya

T1a.

Tambahkan method abstract long adminFee(long amount); ke PaymentMethod, lalu kompilasi.

Error Credit Card

```
public class CreditCardPayment implements RefundablePayment, InstallmentPayment {
                                          ^
  symbol: class RefundablePayment
CreditCardPayment.java:3: error: cannot find symbol
public class CreditCardPayment implements RefundablePayment, InstallmentPayment {
                                                              ^
  symbol: class InstallmentPayment
CreditCardPayment.java:19: error: method does not override or implement a method from a supertype
  @Override
  ^
CreditCardPayment.java:27: error: method does not override or implement a method from a supertype
  @Override
  ^
CreditCardPayment.java:38: error: method does not override or implement a method from a supertype
  @Override
  ^
CreditCardPayment.java:42: error: method does not override or implement a method from a supertype
  @Override
  ^
6 errors
```

Error Bank Transfer

```
public class BankTransferPayment implements PaymentMethod {
                                            ^
  symbol: class PaymentMethod
BankTransferPayment.java:12: error: method does not override or implement a method from a supertype
  @Override
  ^
BankTransferPayment.java:16: error: method does not override or implement a method from a supertype
  @Override
  ^
3 errors
```

Error E-Wallet

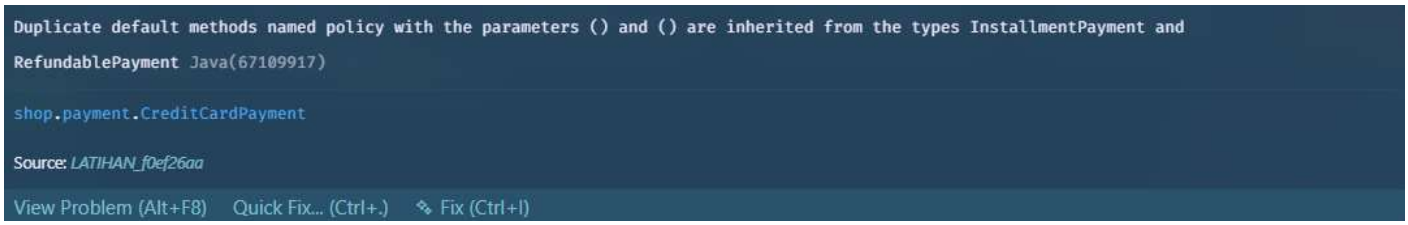
```
EWalletPayment.java:3: error: cannot find symbol
public class EWalletPayment implements RefundablePayment {
                                       ^
  symbol: class RefundablePayment
EWalletPayment.java:18: error: method does not override or implement a method from a supertype
@Override
^
EWalletPayment.java:21: error: method does not override or implement a method from a supertype
@Override
^
EWalletPayment.java:29: error: method does not override or implement a method from a supertype
  @Override
  ^
4 errors
```

Error ShopApp (Anonymous class)

```
PS C:\Users\USER\Documents\College Shit\S3\OOP\Praktikum OOP\LATIHAN\src\shop\payment> cd "c:\Users\USER
um OOP\LATIHAN\src\shop\app\" ; if ($?) { javac ShopApp.java } ; if ($?) { java ShopApp }
ShopApp.java:42: error: cannot find symbol
CreditCardPayment card = new CreditCardPayment("Siti Rahma", 1_000_000);
^
  symbol:   class CreditCardPayment
  location: class ShopApp
ShopApp.java:42: error: cannot find symbol
CreditCardPayment card = new CreditCardPayment("Siti Rahma", 1_000_000);
                             ^
  symbol:   class CreditCardPayment
  location: class ShopApp
ShopApp.java:44: error: cannot find symbol
PaymentMethod voucher = new PaymentMethod() {
^
  symbol:   class PaymentMethod
  location: class ShopApp
ShopApp.java:44: error: cannot find symbol
PaymentMethod voucher = new PaymentMethod() {
                            ^
  symbol:   class PaymentMethod
  location: class ShopApp
ShopApp.java:46: error: method does not override or implement a method from a supertype
    @Override
    ^
ShopApp.java:49: error: method does not override or implement a method from a supertype
    @Override
    ^
42 errors
```

Keempat kelas tersebut mengimplementasikan kelas PaymentMethod. Karena method adminFee bersifat abstract, maka setiap kelas harus mengisi body dari method tersebut.

T1.e



CreditCardPayment mengimplementasikan dua interface yang masing-masing memiliki default method policy(). tetapi dengan isi yang berbeda. Sehingga ia mewarisi dua versi yang bentrok.

T1

1) Mengapa EWalletPayment tidak perlu meng-override policy()?

EWalletPayment hanya mengimplementasikan satu interface saja, yaitu RefundablePayment, dan method policy hanya berasal dari interface itu saja dan body dari method policy dari interface tersebutlah yang memang digunakan untuk kelas ini, sehingga memang tidak perlu diubah/override.

2) Mengapa Java tidak memilih salah satu default secara otomatis?

Mengambil contoh dari konflik diamond sebelumnya, CreditCardPayment mewarisi dua policy() dengan isi berbeda dari dua interface yang sejajar, dan tidak ada alasan yang logis untuk mengutamakan salah satu method. Ini bukan suatu hal yang dapat diselesaikan oleh Java secara “bebas”. Karena itu, Java memaksa untuk menentukan sendiri lewat override. Di dalamnya, versi masing-masing interface tetap bisa dipanggil dengan sintaks NamaInterface.super.policy().

T2

1. Mengapa ada interface dan abstract class, bukan salah satunya saja?

Interface dan abstract class menyelesaikan masalah yang berbeda dan keduanya salng melengkapi.

Interface berfungsi sebagai kontrak yang menyatakan apa yang bisa dilakukan sebuah objek (can-do). Interface bisa diimplementasikan oleh class mana pun, dan satu class boleh mengimplementasikan banyak interface.

Abstract class menyatakan jenis sebuah objek (is-a) dan dapat memuat field, konstruktor, serta method berisi yang dipakai bersama oleh subclass-nya, tetapi sebuah class hanya bisa mewarisi satu abstract class.

Jika hanya memakai interface, setiap class yang mengimplementasikannya harus menulis ulang kode yang sama, misalnya alur kerja dan data bersama. Jika hanya

memakai abstract class, class yang sudah mewarisi induk lain tidak bisa ikut memenuhi kontrak tersebut, dan kode yang memakainya jadi terikat pada satu hierarki tertentu saja.

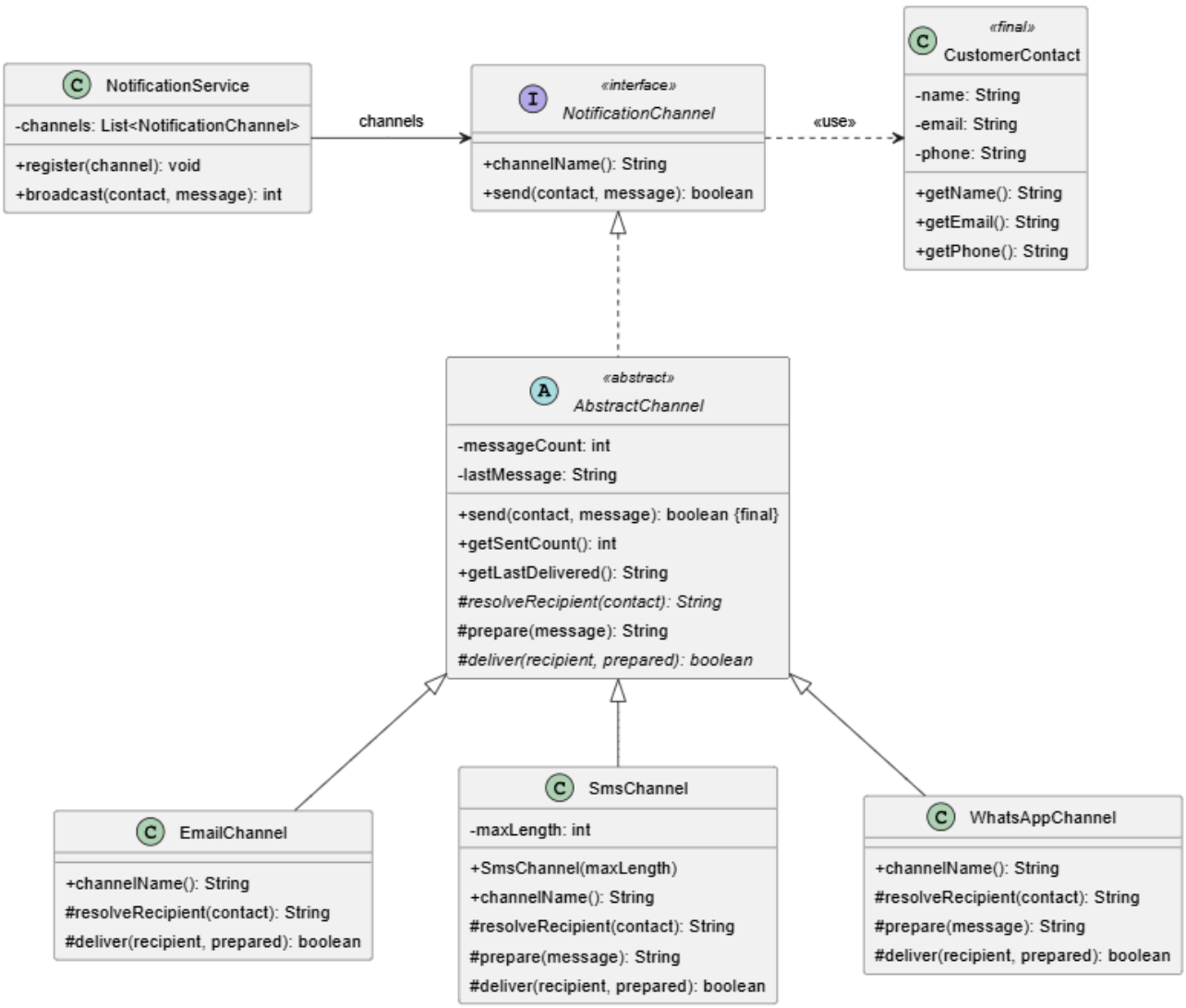
2. Apa akibatnya jika send(...) tidak dibuat final?

Sub-class bisa meng-override send dan mengubah urutan langkahnya. Tidak ada juga bagian dari sub-class yang secara urgent membutuhkan langkah dari method send untuk diubah, karena itu method send dikunci dengan diberikan final.

3. Mengapa NotificationService menyimpan List<NotificationChannel>, bukan List<AbstractChannel>?

Objek itu adalah anonymous class yang mengimplementasikan NotificationChannel, bukan turunan AbstractChannel, sehingga tidak bisa masuk ke list bertipe AbstractChannel.

UML Diagram



T3

No.	Lokasi	Masalah	Dampak	Perbaikan usulan
1	interface *StoreOperations*	Memaksakan method yang mungkin tidak diperlukan pada suatu kelas. Pada kasus ini yaitu Voucher yang tidak bisa di-refund dan satuan berat tidak diperlukan.	Program crash saat dijalankan	Memisahkan interface refund dan shippable menjadi interfacenya masing masing
2	Abstract class Courier + JNECourier (sub-class Courier)	Hanya abstrak, tidak ada atribut, field, bahkan method abstract satupun. Method cost tidak final, sehingga dapat diubah oleh subclass	Jika terjadi misinput pada jenis layanan kurir, akan membuat ongkir bernilai 0 (dan juga strukturnya tidak elegan)	Mengubah logika kelas Courier sepenuhnya; menambahkan method abstrak ratePerKg
3	Kelas Address	Address dapat diubah dan satu objeknya dipakai bersamaan untuk dua pesanan berbeda.	Alamat pengiriman bisa saja berubah tanpa disadari saat satu objek alamat dipakai oleh beberapa pesanan sekaligus.	Address dibuat final dengan field private final dan tanpa setter
4	Kelas Cart.	Terdapat inner class (Item) yang tidak static, padahal tidak membaca objek dari luas kelasnya.	Pemborosan memori untuk suatu toko yang menjual ribuan jenis barang	Menjadikan kelas item yang merupakan inner class menjadi static, dan mengubah cara instansiai objek pada kelas main
5	interface *Discount*	Tiap ada jenis diskon baru, perlu dilakukan perubahan pada method utama.	Jika tidak ada satu cabang yang menghandle dengan benar, diskon baru itu tidak berlaku dan pelanggan dikenai harga penuh tanpa ada error.	Membuat agar tiap jenis diskon memberikan persentase diskon yang sesuai melalui method abstract

T4

Tabel 1

Pertanyaan desain	**Abstract class atau interface?**	**Alasan**	**Contoh dari proyek**
Subclass berbagi field dan konstruktor	Abstract class	Interface tidak punya field instance dan konstruktor, hanya abstract class yang punya.	Kelas Product
Satu class perlu beberapa kemampuan sekaligus	Interface	Class dapat meng-implements banyak interface sekaligus, tetapi hanya bisa extend satu kelas	CreditCardPayment yang mengimplementasikan RefundablePayment dan InstallmentPayment
Rumus bersama yang tidak boleh diubah subclass	Abstract class	Abstract class bisa memuat method final yang tidak dapat diubah oleh sub-classnya	ShippingService.calculateCost bersifat final
Kontrak yang perlu ditambah method baru di kemudian hari	Interface	Default method pada interface digunakan untuk memberi isi bawaan, sehingga class lama tidak perlu diubah.	PaymentMethod.adminFee dibuat default return 0

Tabel 2

Kondisi	**Static nested, inner, atau anonymous?**	**Alasan**	**Contoh dari proyek**
Class pembantu tidak membaca data objek luar	Static nested	Tidak butuh referensi ke objek luar, dan bisa dibuat tanpa objek	Order.Line (product, quantity, subtotal)
Objek harus membaca field objek luarnya	Inner class	Menyimpan referensi ke objek luar, sehingga bisa dibaca.	Order.Receipt membaca orderId dan shippingAddress milik Order
Implementasi sekali pakai, tidak perlu nama	Anonymous class	Cukup dipakai di satu tempat, tidak perlu diberi nama ataupun dipakai ulang	Voucher PaymentMethod di ShopApp Log NotificationChannel di TugasApp

SCREENSHOTS

```
-- L4 - final method & final class
  [LULUS] L4.1 ShippingService.calculateCost(int) adalah final method
  [LULUS] L4.2 Address final class, semua field private final, tanpa setter

-- L5 - Static nested, inner, dan anonymous class
  [LULUS] L5.1 Order.Line static nested; Order.Receipt inner (non-static)
  [LULUS] L5.2 Receipt.toText() memuat id pesanan, total barang, dan kota tujuan
  [LULUS] L5.3 ShopApp memakai anonymous class yang mengimplementasikan PaymentMethod

=== HASIL: 23 / 23 pemeriksaan lulus ===
```

Self Check

```
  [LULUS] T2.4 EmailChannel menolak kontak tanpa email yang valid
[WA to +6281298765432] [Toko UMKM] Pesanan dikirim
  [LULUS] T2.5 NotificationService.broadcast(...) mengembalikan jumlah kanal yang berhasil

=== HASIL: 10 / 10 pemeriksaan lulus ===
```

Tugas Check

```
Nama: Muhammad Yasin Habiburrahman
NIM: 2511532016

== T1: Total dengan biaya admin (Ekspres) ==
E-Wallet Okane: Rp261000
Transfer Abank: Rp264000
Kartu Kredit (Suheng): Rp263610

Kebijakan Kartu Kredit: Refund maksimal 7 hari; Cicilan maksimal 12 bulan
Kebijakan E-Wallet: Refund maksimal 7 hari

== T2: Notifikasi status pesanan ==
[SMS to 085254192511] Pesanan Anda sudah dikirim dengan kurir ekspres. Nomor resi akan muncul di dalam aplikasi dalam 1x24 jam. Jika pe...
[WA to 085254192511] [Toko UMKM] Pesanan Anda sudah dikirim dengan kurir ekspres. Nomor resi akan muncul di dalam aplikasi dalam 1x24 jam. Jika pesanan belum s
ampai dalam 3 hari kerja, hubungi layanan pelanggan kami lewat aplikasi. Terima kasih telah berbelanja produk-produk negara asing dari toko kami!
[EMAIL to Suhengers@gmail.com] Pesanan Anda sudah dikirim dengan kurir ekspres. Nomor resi akan muncul di dalam aplikasi dalam 1x24 jam. Jika pesanan belum sam
pai dalam 3 hari kerja, hubungi layanan pelanggan kami lewat aplikasi. Terima kasih telah berbelanja produk-produk negara asing dari toko kami!
log Suheng: Pesanan Anda sudah dikirim dengan kurir ekspres. Nomor resi akan muncul di dalam aplikasi dalam 1x24 jam. Jika pesanan belum sampai dalam 3 hari ke
rja, hubungi layanan pelanggan kami lewat aplikasi. Terima kasih telah berbelanja produk-produk negara asing dari toko kami!
Terkirim ke Suheng lewat 4 kanal
[WA to +6285354192511] [Toko UMKM] Pesanan Anda sudah dikirim dengan kurir ekspres. Nomor resi akan muncul di dalam aplikasi dalam 1x24 jam. Jika pesanan belum
 sampai dalam 3 hari kerja, hubungi layanan pelanggan kami lewat aplikasi. Terima kasih telah berbelanja produk-produk negara asing dari toko kami!
log Sugeng: Pesanan Anda sudah dikirim dengan kurir ekspres. Nomor resi akan muncul di dalam aplikasi dalam 1x24 jam. Jika pesanan belum sampai dalam 3 hari ke
rja, hubungi layanan pelanggan kami lewat aplikasi. Terima kasih telah berbelanja produk-produk negara asing dari toko kami!
Terkirim ke Sugeng lewat 2 kanal
PS C:\Users\USER\Documents\College Shit\SJ\OOP\Praktikum OOP\LATIHAN>
```

TugasApp